

CAPSTONE PROJECT REPORT

Learning Community Analytics using NLP Topics and Knowledge Diffusion Networks

A case study of the CS50 Stack Exchange learning community, 2019–2026

Submitted by

Prateek Chauhan

Project guide: _____

Department / Institute: _____

September 2026

Abstract

Online learning communities let thousands of learners ask questions and help each other, but it is hard for instructors and platforms to see what learners struggle with, who carries the community, and how knowledge actually spreads. This project builds an end-to-end analytics system for one such community, the CS50 Stack Exchange site, using 4,000 questions, 4,253 answers and 7,168 comments from 3,003 users (2019–2026).

Discussion topics were discovered with two methods. BERTopic (sentence embeddings with UMAP and HDBSCAN) outperformed an LDA baseline on every quality measure: NPMI coherence 0.31 vs 0.24, topic diversity 0.87 vs 0.73, and agreement with the course's own problem-set tags (NMI) 0.56 vs 0.38. A difficulty index built from unanswered rate, waiting time and learner confusion ranked *pair / tideman / sort* (Tideman, Week 3) as the hardest topic.

A helper-to-learner knowledge network of 2,891 users showed extreme concentration: the top 1% of users gave 81% of all help. Louvain found 16 communities of ten or more users (modularity 0.60), and ExpertiseRank agreed closely with accepted answers (top-20 overlap 85%). A time-shuffle test found a modest but significant social-exposure effect across topics (Stouffer $z = 2.68$, $p = 0.004$), concentrated in later-course topics. Independent Cascade simulation matched observed time-respecting reach (Spearman $\rho = 0.86$).

Two applications were built on these results. A Random Forest predicts whether a new question will be answered (ROC-AUC 0.675 on future questions), and a hybrid expert finder places a real answerer in its top-10 list for 81% of future questions, against 70% for a popularity baseline. All data and results are stored in PostgreSQL and explored through an interactive Streamlit dashboard.

Keywords: learning analytics, topic modeling, BERTopic, social network analysis, knowledge diffusion, influence maximisation, expert recommendation, Stack Exchange, CS50.

Table of Contents

Abstract.....	2
Table of Contents.....	3
1. Introduction	5
1.1 Background	5
1.2 Problem statement	5
1.3 Objectives.....	5
1.4 Contributions	5
1.5 Report organisation	6
2. Literature Review	7
2.1 Topic modeling.....	7
2.2 Networks in Q&A communities	7
2.3 Diffusion and social influence	7
2.4 Answer prediction and expert recommendation	7
2.5 Research gap	7
3. Data and System Architecture	8
3.1 Data source	8
3.2 Database design	8
3.3 System architecture	9
4. Methodology.....	10
4.1 Text preprocessing.....	10
4.2 Topic modeling.....	10
4.3 Knowledge network	10
4.4 Knowledge diffusion	11
4.5 Answer prediction.....	11
4.6 Expert recommendation	11
5. Results.....	13
5.1 Discussion topics.....	13
5.2 The knowledge network	15
5.3 Knowledge diffusion	17
5.4 Answer prediction.....	20
5.5 Expert recommendation	21
5.6 Interactive dashboard	22
6. Discussion.....	23
6.1 What the results mean	23
6.2 Limitations.....	23
6.3 Lessons learned.....	23

7. Conclusion and Future Work	24
Future work.....	24
References	25
Appendix A. Running the System.....	26
Tools and libraries.....	26

1. Introduction

1.1 Background

Question-and-answer communities have become an important part of how people learn programming. Beginners who are stuck on an exercise post a question; more experienced learners, teaching assistants and alumni answer. Over time these interactions form a community with its own experts, sub-groups and shared knowledge. Harvard's CS50, one of the most widely taken introductory computer science courses, runs such a community on Stack Exchange, where learners from around the world discuss the course's problem sets.

Everything that happens in the community is recorded as text (what people ask and answer) and as interactions (who answers whom, and when). Together these records can show what learners find difficult, who holds the community together, and how knowledge moves from person to person. Turning the records into that picture needs two families of methods: natural language processing (NLP) for the text, and network science for the interactions.

1.2 Problem statement

Instructors and platform designers have little visibility into their learning communities. They cannot easily answer: Which topics do learners struggle with most? Which members provide most of the help, and what happens if they leave? Does knowledge spread through personal contact, or does everyone learn independently? Which new questions are at risk of never being answered, and who could answer them?

1.3 Objectives

1. Collect a complete, reproducible dataset of a real learning community and store it in a relational database (PostgreSQL).
2. Discover discussion topics with NLP, compare a classical model (LDA) with a transformer-based model (BERTopic), and validate the topics against the course syllabus.
3. Build the knowledge network of who helps whom, identify experts, bridges and communities, and validate the network measures against independent signals.
4. Test whether knowledge spreads through social contact, and simulate diffusion with the Independent Cascade and Linear Threshold models, including influence maximisation.
5. Build two practical applications: an answer predictor and an expert recommender, both evaluated on future data.
6. Present all results in an interactive dashboard.

1.4 Contributions

- A leakage-aware, fully reproducible pipeline from raw API data to dashboard, running on either PostgreSQL or flat files.

- A fair comparison of LDA and BERTopic on a shared vocabulary, using coherence, diversity and agreement with course tags.
- A topic difficulty index that combines unanswered rate, answer delay and a lexicon-based confusion score.
- A statistical test of social influence (time-shuffle test with Stouffer combination) alongside simulated diffusion, with the simulation validated against observed time-respecting reach.
- Honest reporting of negative results, such as the absence of a learner-to-helper effect, which were first masked by bugs that the validation process caught.

1.5 Report organisation

Chapter 2 reviews related work. Chapter 3 describes the data and system architecture, and Chapter 4 the methods. Chapter 5 presents the results, Chapter 6 discusses their meaning and limitations, and Chapter 7 concludes. The appendix explains how to run the system.

2. Literature Review

2.1 Topic modeling

Latent Dirichlet Allocation (LDA) [1] models each document as a mixture of topics and each topic as a distribution over words. It is the standard baseline but relies on bag-of-words counts, which struggle with short, technical texts. BERTopic [2] instead embeds each document with a pretrained sentence transformer [3], reduces the embeddings with UMAP [4], clusters them with HDBSCAN [5], and describes each cluster with class-based TF-IDF. Topic quality is commonly measured with normalised pointwise mutual information (NPMI) coherence [6] and topic diversity [7].

2.2 Networks in Q&A communities

Zhang, Ackerman and Adamic [8] modelled a Java help forum as a network where askers point to answerers and showed that a PageRank-style ExpertiseRank identifies experts better than counting answers. Community detection with the Louvain method [9] reveals groups of users who mostly interact with each other. Studies of Stack Overflow and MOOC forums repeatedly report that a small core of users provides most answers [10].

2.3 Diffusion and social influence

Kempe, Kleinberg and Tardos [11] formalised influence maximisation under the Independent Cascade (IC) and Linear Threshold (LT) models and showed that greedy seed selection is within $(1 - 1/e)$ of optimal; the CELF algorithm [12] makes it practical through lazy evaluation. Separating influence from similarity (homophily) in observational data is difficult; Anagnostopoulos, Kumar and Mahdian [13] proposed the shuffle test, which compares the observed timing of adoptions with randomly permuted timings.

2.4 Answer prediction and expert recommendation

Predicting whether questions will be answered, and routing them to suitable experts, have been studied on Stack Overflow using text, asker history and temporal features [14]. Expert recommenders combine content similarity with authority and activity signals [15]. A recurring lesson is that evaluation must respect time: training on the past and testing on the future avoids optimistic estimates.

2.5 Research gap

Most studies analyse topics, networks or diffusion separately and rarely validate simulated diffusion against what actually happened. This project combines all three on one learning community, validates each component against independent evidence (course tags, reputation, observed reach), and turns the results into tools that instructors could use.

3. Data and System Architecture

3.1 Data source

The data comes from the public Stack Exchange API (v2.3) for the site cs50.stackexchange.com. The collector downloads the newest questions with their bodies, then all answers and comments on those questions. Without an API key the API serves only 25 pages per query, so the collector walks backwards through time in date windows. The final dataset covers 2019-05-13 to 2026-09-18.

Table 1. Dataset summary

Item	Count
Questions	4,000
Answers	4,253
Comments	7,168
Users	3,003
Interactions (answer or comment → post owner)	10,477
Questions answered (Stack Exchange is_answered)	66.1%
Questions with an accepted answer	51.6%
Questions containing code	80%
Median time to first answer	4.0 hours

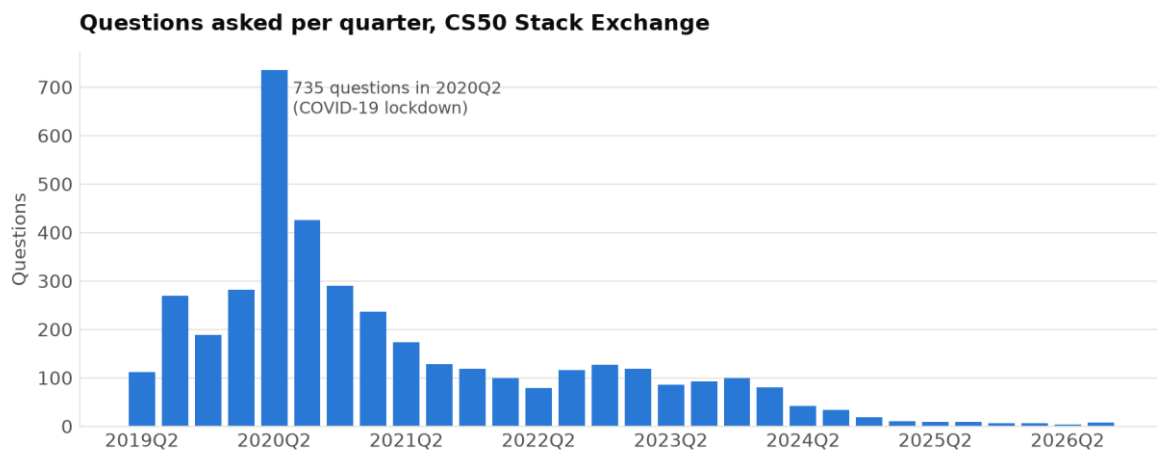


Figure 1. Questions per quarter. Activity peaked in 2020Q2 (735 questions) during the COVID-19 lockdown and has declined since.

3.2 Database design

The raw data is stored in PostgreSQL in five tables with primary and foreign keys: **users**, **questions** (→ users), **answers** (→ questions, users), **comments** (→ users), and **interactions**, the edge list of the knowledge network. After the analysis, eleven result tables (topics, question topics, user metrics, help edges, topic trends, model evaluations and more) are loaded next to them, so the whole project can be queried with SQL in pgAdmin. Eleven ready-made analysis queries are provided, and every query was checked with the PostgreSQL parser.

3.3 System architecture

The system is a pipeline of independent Python modules that read from PostgreSQL when a connection is configured and from CSV files otherwise:

1. **collect_data**: Stack Exchange API → CSV
2. **load_to_postgres**: CSV and results → PostgreSQL
3. **preprocess**: text cleaning and features
4. **topic_model**: LDA and BERTopic
5. **network**: knowledge network and centrality
6. **diffusion**: exposure tests and IC/LT simulation
7. **predict** and **recommender**: the two applications
8. **figures** and the **Streamlit dashboard**: presentation

A single command (`run_pipeline.py`) runs the whole analysis in about 10 minutes on a laptop CPU. All random steps use fixed seeds; two full runs produced identical topic results.

4. Methodology

4.1 Text preprocessing

Question bodies are HTML. Code blocks were removed from the text (their line count kept as a feature), HTML was stripped, and URLs were deleted. The title and body form the natural-language text used for sentence embeddings. For word-based models, the text was lower-cased, tokenised, lemmatised with WordNet, and filtered with English stopwords plus about 150 domain stopwords such as “cs50”, “help” and “error” that appear in almost every question.

Each question also received structural features (length, code lines, number of tags, question marks), an error flag (compiler errors, segmentation faults, valgrind output), a check50-failure flag (CS50's grader prints “:(” for each failed test), a VADER sentiment score, and a confusion score counting phrases such as “stuck”, “don't understand” and “what am I doing wrong”, normalised by length.

4.2 Topic modeling

LDA. A bag-of-words matrix of unigrams and bigrams (minimum 5 documents, maximum 40% of documents) was built. LDA was fitted for $k = 6$ to 20, and k was chosen by NPMI coherence $\times$ diversity, which selected $k = 20$.

BERTopic. Each question was embedded with the all-MiniLM-L6-v2 sentence transformer, reduced to five dimensions with UMAP and clustered with HDBSCAN (minimum cluster size 30). Near-duplicate clusters, such as several “recover” clusters, were merged to 19 topics, comparable with LDA, and outlier questions were assigned to the nearest topic in embedding space.

Fair evaluation. BERTopic builds its vocabulary from one merged document per topic, so its top words can be rare phrases that never co-occur in single questions, which unfairly lowers NPMI. Both models were therefore scored on the same reference vocabulary: words appearing in at least five questions. Three metrics were used: NPMI coherence of the top 10 words, topic diversity of the top 25 words, and normalised mutual information (NMI) between topics and each question's course unit, mapped from its problem-set tags (for example tideman $\rightarrow$ Week 3, finance $\rightarrow$ Week 8–9).

Difficulty index. For each topic, the unanswered rate, the log of the median hours to the first answer, and the mean confusion score were z-standardised and averaged.

4.3 Knowledge network

A directed, weighted graph was built with an edge from helper to learner for every answer, and for every comment written by someone other than the thread's asker. The second rule was added after validation showed that askers' replies (“thanks, still broken”) were being counted as help and inflating reciprocity. For each user the following measures were computed:

- in- and out-degree and strength (number of interactions)
- ExpertiseRank, which is PageRank on learner $\rightarrow$ helper links [8]
- betweenness, clustering coefficient and k-core number

- Louvain community and participation coefficient

Rule-based roles (Core expert, Bridge, Helper, Active participant, Peripheral learner) summarise these measures. The network measures were validated against Stack Exchange reputation and accepted answers, which the network does not use, with Spearman correlation and top-20 overlap among users who answered at least once.

4.4 Knowledge diffusion

Social exposure test. A user adopts a topic the first time they ask or answer in it. An adopter counts as exposed if, before adopting, they interacted with someone who had already adopted the topic. The observed exposed share was compared with 200 shuffles in which adoption times were permuted among the same adopters, keeping the network and interaction times fixed [13]. The per-topic z-scores were combined with Stouffer's method.

Learner → helper transition. A chi-square test compared learners whose first question was answered with those whose first question was not, on whether they later answered someone else's question. Self-answers were excluded.

Simulation. Knowledge flows from helper to learner. In IC, each of the w interactions on an edge passes knowledge with probability $\beta = 0.1$, so $p = 1 - 0.9^w$. In LT, the influence weights are w divided by the learner's total incoming help. Seeds were chosen by CELF greedy optimisation [12] among the 150 most active helpers and compared with top- k by out-degree, ExpertiseRank, betweenness, PageRank, and random seeds, using 300 Monte Carlo runs per evaluation.

Validation. Each candidate's single-seed IC spread was compared with their observed time-respecting reach: the number of users reachable along helper → learner interactions with increasing timestamps.

4.5 Answer prediction

The target is Stack Exchange's `is_answered` flag. To avoid leakage, only information available when the question is posted was used: text structure, sentiment and confusion, topic and course unit, the asker's earlier questions, answers and answered rate, community activity in the previous 30 days, the topic's earlier answered rate, and the posting hour and weekday. The data was split in time: the oldest 3,168 questions (until 2022-08-16) for training and the newest 792 for testing. Logistic Regression, Random Forest and XGBoost were compared with a majority-class baseline; feature importance was measured by permutation.

4.6 Expert recommendation

For a new question, the 50 most similar past questions were found by cosine similarity of sentence embeddings, or TF-IDF as a comparison. Each helper's content score is the sum of similarities of those questions they answered, with accepted answers counted double. The hybrid score blends content (0.6), authority, $\log(1 + \text{answers})$ (0.2), and recency, $\exp(-\text{days since last answer} / 365)$ (0.2). Helpers were learned from the oldest 80% of questions; for each newer question, the evaluation checks

whether a person who actually answered it appears in the top k (Hit@ k) and reports mean reciprocal rank (MRR).

5. Results

5.1 Discussion topics

Table 2 compares the two topic models. BERTopic is better on all three measures, and the largest gain is in agreement with the course structure (NMI 0.56 vs 0.38). Its topics map cleanly onto CS50 problem sets: speller, filter, recover, caesar, plurality/runoff, finance, DNA and tideman each form their own topic, many with more than 85% of their tagged questions from a single course unit.

Table 2. Topic model comparison (shared reference vocabulary)

Model	Topics	NPMI ↑	Diversity ↑	NMI vs course tags ↑
LDA	20	0.243	0.730	0.384
BERTopic	19	0.310	0.865	0.561

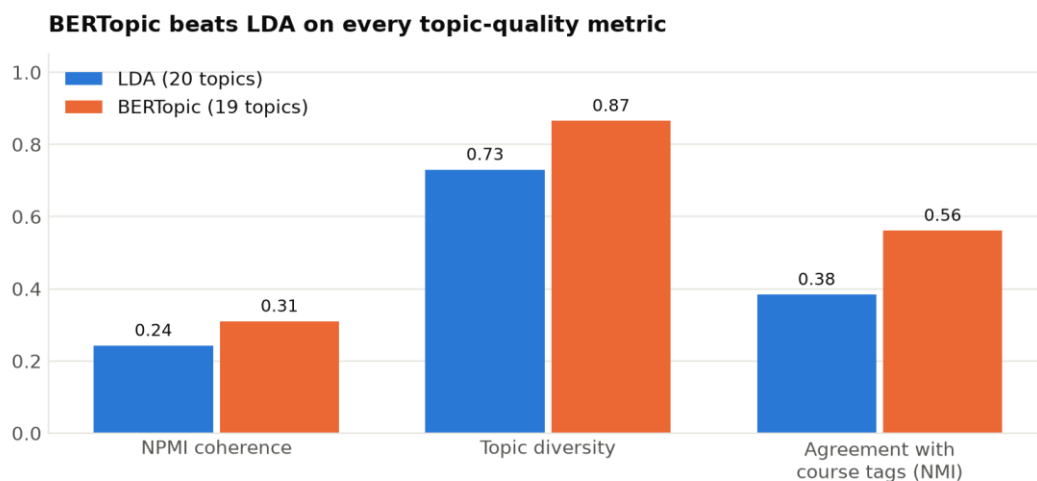


Figure 2. BERTopic outperforms LDA on coherence, diversity and agreement with the course's problem-set tags.

Figure 3 ranks topics by the difficulty index. **pair / tideman / sort** (Week 3 – Algorithms) is the hardest: 45% of its questions stayed unanswered, with a median wait of 12.7 hours. Tideman is widely known among CS50 students as the hardest problem set, which gives an independent check on the index. The web-development topics (Flask, registration and Finance) follow. Early, well-structured exercises such as caesar and speller are the easiest, because they get answered quickly.

Topic difficulty index
(unanswered rate + time to answer + learner confusion)

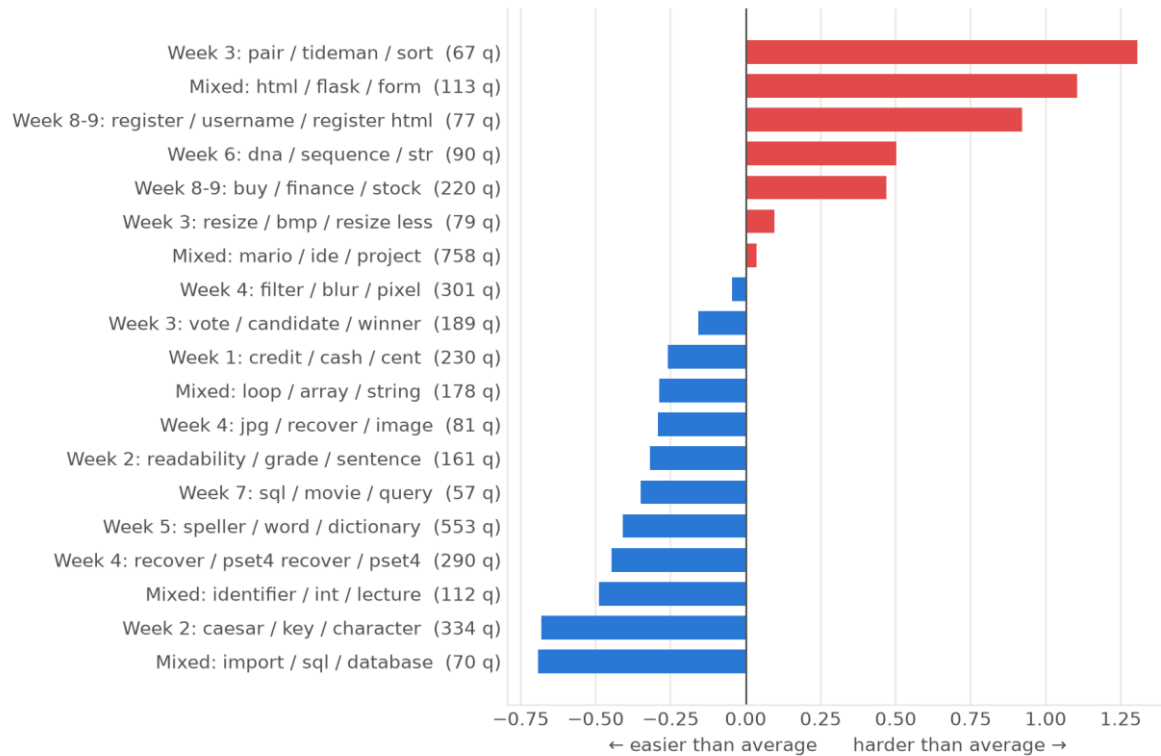


Figure 3. Topic difficulty index. Red topics are harder than average; “Mixed” marks topics without a dominant course unit.

Table 3. All BERTopic topics

Topic (top words)	Course unit	Questions	Unanswered	Difficulty
mario / ide / project	Mixed	758	42%	0.04
speller / word / dictionary	Week 5 – Data structures	553	29%	-0.41
caesar / key / character	Week 2 – Arrays	334	25%	-0.68
filter / blur / pixel	Week 4 – Memory	301	29%	-0.04
recover / pset4 recover / pset4	Week 4 – Memory	290	27%	-0.45
credit / cash / cent	Week 1 – C basics	230	27%	-0.26
buy / finance / stock	Week 8-9 – Web / Flask	220	43%	0.47
vote / candidate / winner	Week 3 – Algorithms	189	34%	-0.16
loop / array / string	Mixed	178	33%	-0.29
readability / grade / sentence	Week 2 – Arrays	161	34%	-0.32
html / flask / form	Mixed	113	56%	1.10
identifier / int / lecture	Mixed	112	30%	-0.49
dna / sequence / str	Week 6 – Python	90	44%	0.50
jpg / recover / image	Week 4 – Memory	81	20%	-0.29
resize / bmp / resize less	Week 3 – Algorithms	79	22%	0.10
register / username / register html	Week 8-9 – Web / Flask	77	44%	0.92

Topic (top words)	Course unit	Questions	Unanswered	Difficulty
import / sql / database	Mixed	70	44%	-0.69
pair / tideman / sort	Week 3 – Algorithms	67	45%	1.30
sql / movie / query	Week 7 – SQL	57	35%	-0.35

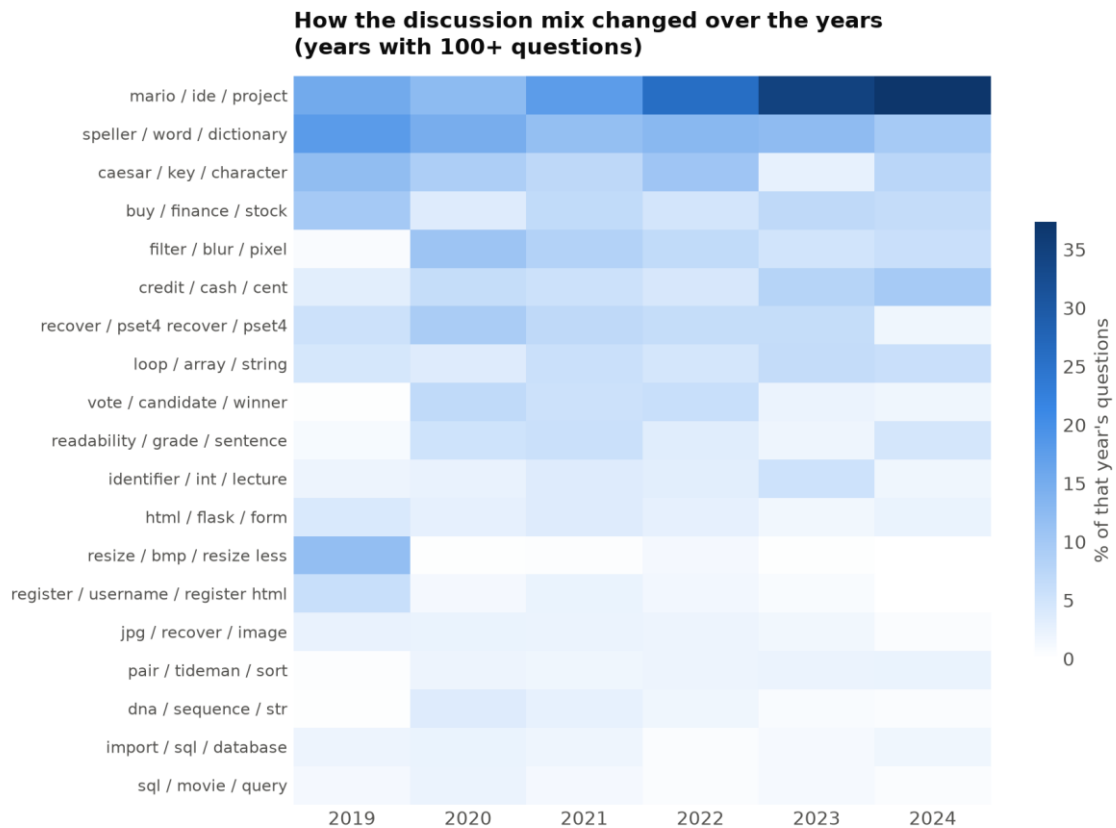


Figure 4. Share of each year's questions by topic. General setup and tooling questions (mario / ide / project) grow as the course moved to new tools such as VS Code and Codespaces.

5.2 The knowledge network

The help network has 2,891 users and 4,245 helper → learner links carrying 6,930 interactions. It is extremely sparse (density 0.0005) and almost fully connected (93% of users are in one component). Only 30% of users ever help someone, and help is highly concentrated: the top 1% of users give 81% of all help and the top 10% give 92% (Gini 0.96). Reciprocity is only 0.035, meaning help flows one way, from experts to learners, and degree assortativity is negative (-0.30): experts mostly help newcomers rather than each other.

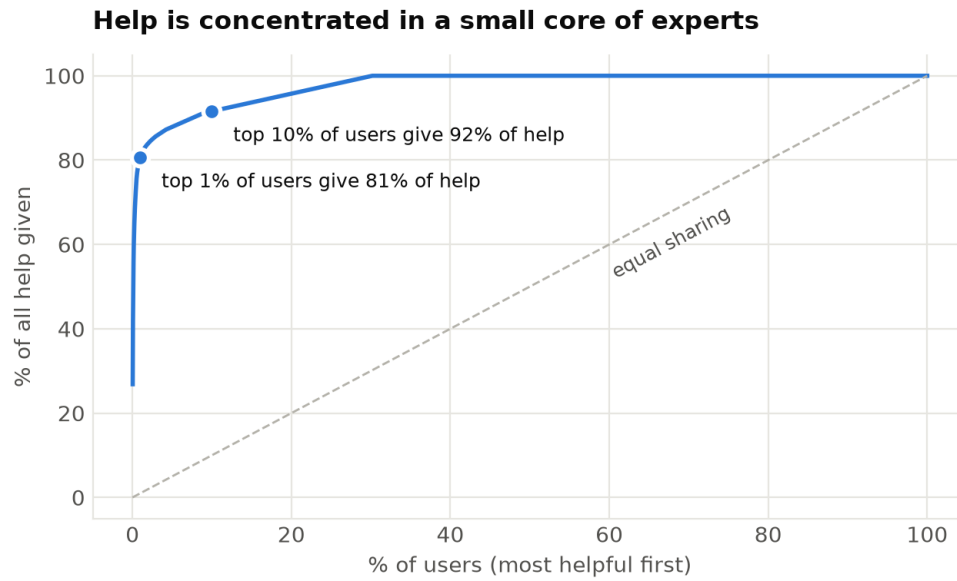


Figure 5. Cumulative share of help given. A small core of experts carries the community.

**Knowledge network core: top 150 helpers and the learners they help
(node size = help given)**

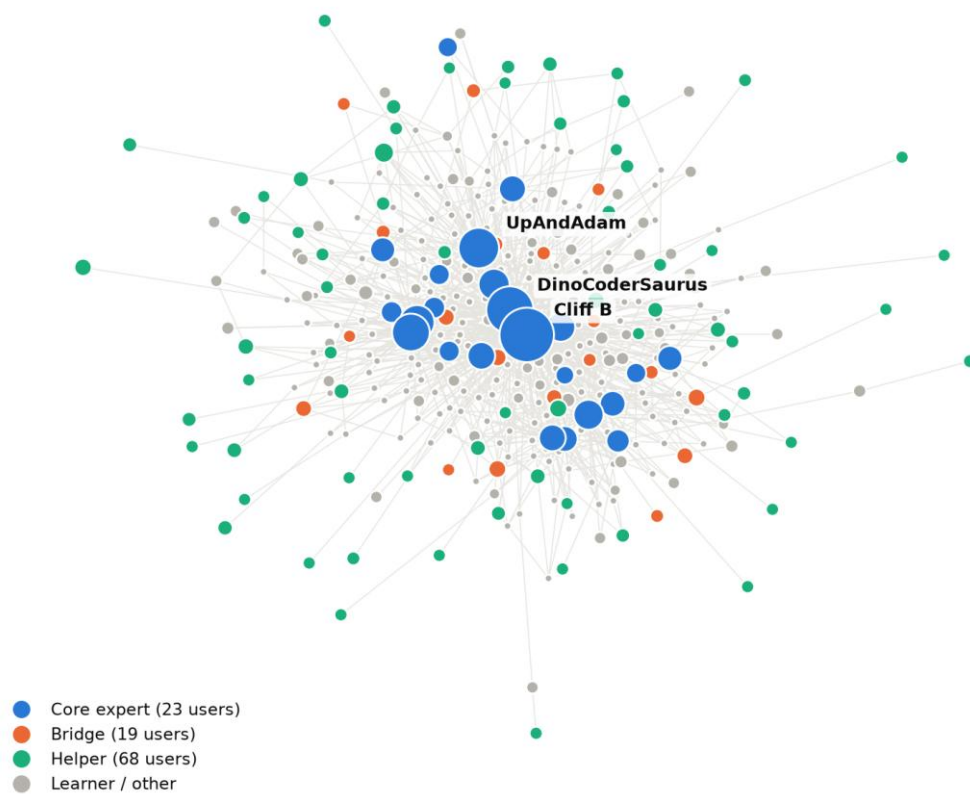


Figure 6. The core of the knowledge network, coloured by role. Core experts sit at the centre; bridges connect otherwise separate groups.

Louvain found 118 communities, 16 of them with ten or more users (modularity 0.60). Each large community forms around one or two core experts. Role assignment identified 23 core experts, 19 bridges and 68 regular helpers.

Table 4. Validation of network measures among 821 users who answered at least once

Measure	p with accepted answers	Top-20 overlap	p with reputation	Top-20 overlap
expertise rank	0.15	85%	0.23	70%
out strength	0.21	95%	0.25	70%
betweenness	0.31	85%	0.41	70%
core number	0.35	70%	0.37	50%
pagerank	0.20	40%	0.30	35%

ExpertiseRank and out-strength identify the same people as the community's own quality signals: 85% of ExpertiseRank's top 20 are also in the top 20 by accepted answers. Rank correlations over all helpers are moderate (0.15–0.41), because reputation is earned across the whole site history while the network covers only this sample.

The community peaked in 2020 and shrank afterwards

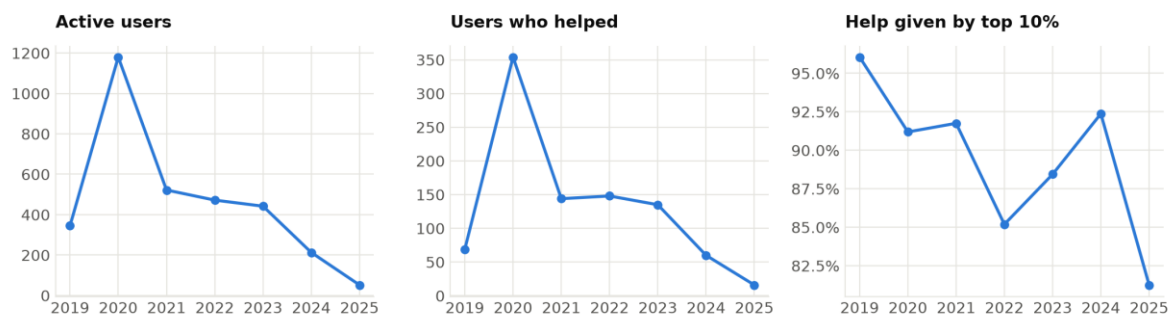


Figure 7. The network over time: activity peaked in 2020, while the share of help given by the top 10% stayed high throughout.

5.3 Knowledge diffusion

Social exposure. Across all topics combined there is a modest but significant exposure effect (Stouffer $z = 2.68$, $p = 0.004$): on average 31.4% of adopters had met an earlier adopter before joining a topic, against 27.1% expected by chance. The effect is significant individually for 3 of 19 topics (buy / finance / stock, dna / sequence / str, sql / movie / query), all from the later part of the course. A plausible reading is that learners who reach the advanced problem sets are more embedded in the community, so peer contact matters more there. For early topics, where most learners arrive independently, adoption timing is consistent with chance.

Table 5. Exposure test, six most significant topics

Topic	Adopters	Observed	Chance	z	p
dna / sequence / str	116	48.7%	29.3%	3.51	0.005
sql / movie / query	79	48.7%	29.4%	2.77	0.005

Topic	Adopters	Observed	Chance	z	p
buy / finance / stock	197	42.4%	27.6%	1.67	0.030
register / username / register html	77	44.7%	31.5%	1.57	0.060
import / sql / database	86	32.9%	24.4%	1.51	0.090
speller / word / dictionary	496	34.5%	25.6%	1.19	0.144

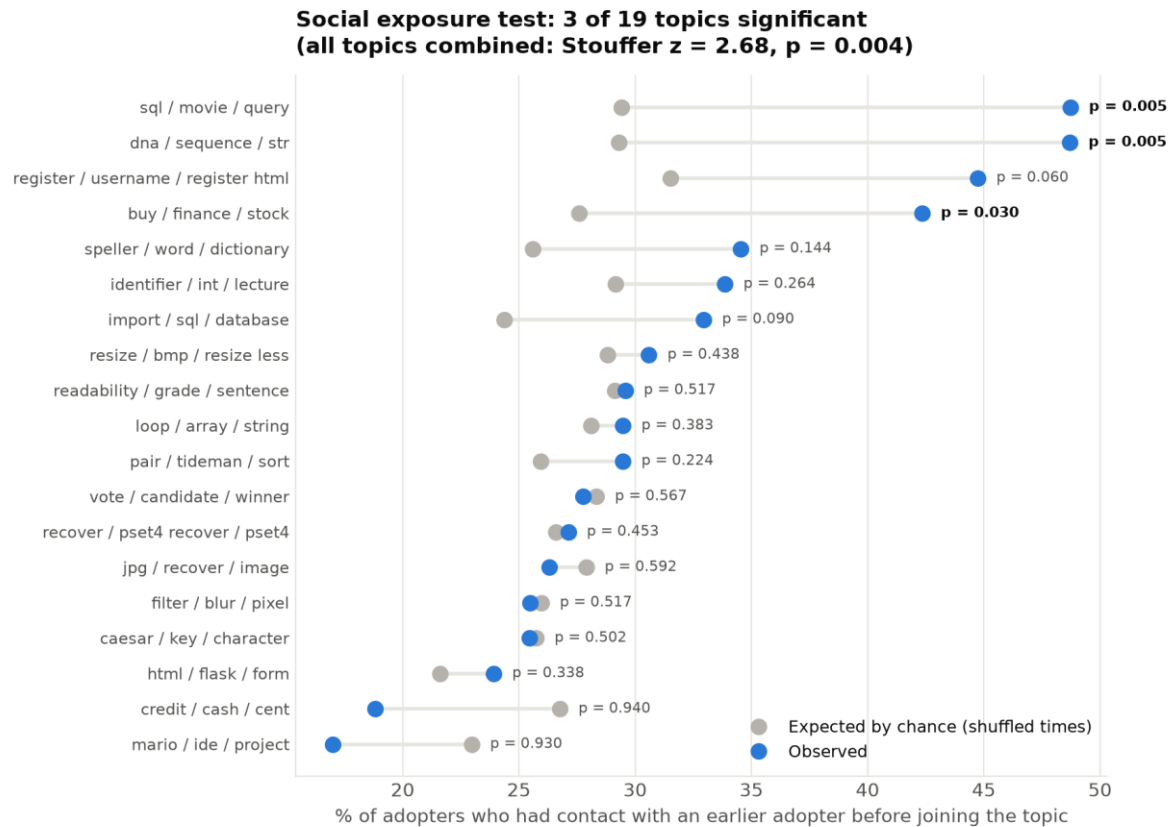


Figure 8. Observed vs chance exposure to earlier adopters for every topic.

Learners becoming helpers. Of 2,374 learners, 125 (5.3%) later answered someone else's question, with a median of 4 days after their first question. Whether their first question was answered made no significant difference (5.2% vs 6.0%, χ^2 p = 0.73). This is an honest negative result: an earlier version of the analysis reported a strong effect, but it came from counting self-answers as help, which the validation step caught.

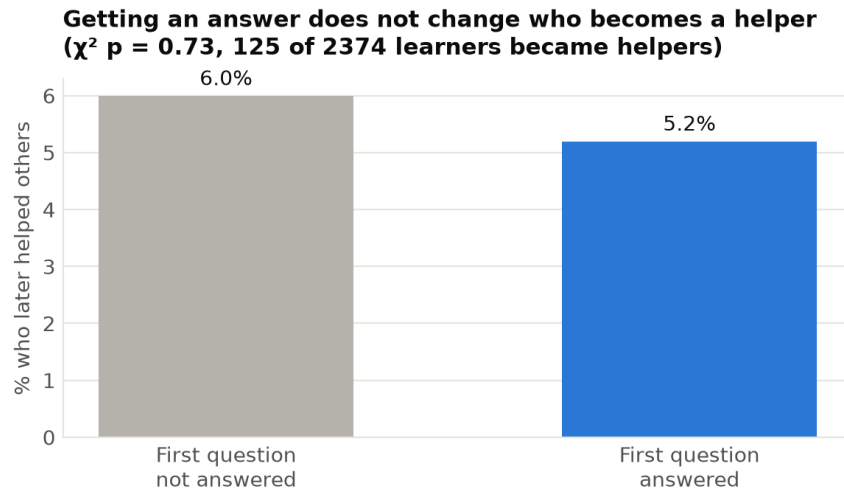


Figure 9. Share of learners who later helped others, by whether their first question was answered.

Influence maximisation. With ten seed users, knowledge reaches about 15% of the network under IC and 68% under LT, against under 1% for random seeds. Simple centrality heuristics (ExpertiseRank, out-degree, betweenness) come within a fraction of a percentage point of CELF. The network is so centralised that the most active experts are close to the best possible seeds, so a platform can find its key spreaders without expensive optimisation.

Table 6. Share of users reached with ten seed users

Seed strategy (k = 10)	IC reach	LT reach
CELF greedy	14.6%	67.6%
Top ExpertiseRank	14.4%	68.1%
Top out-degree	14.4%	67.8%
Top betweenness	14.5%	68.2%
Top PageRank	13.5%	64.5%
Random	0.8%	0.9%

Influence maximisation: which seed users spread knowledge furthest

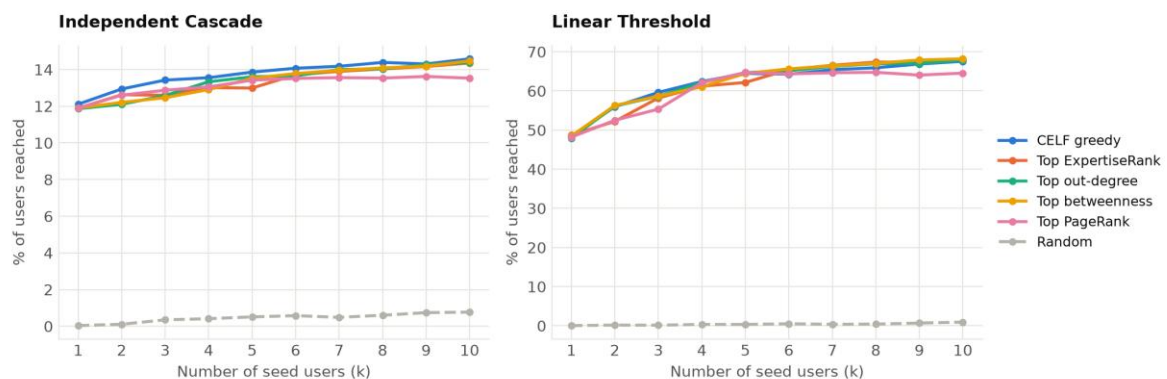


Figure 10. Spread as a function of the number of seeds, for both diffusion models.

Validation. The simulated single-seed IC spread of each of the 150 most active helpers correlates strongly with their observed time-respecting reach (Spearman $\rho = 0.86$). The simulation reproduces the ordering of real spreaders better than raw out-strength ($\rho = 0.65$) or betweenness ($\rho = 0.54$).

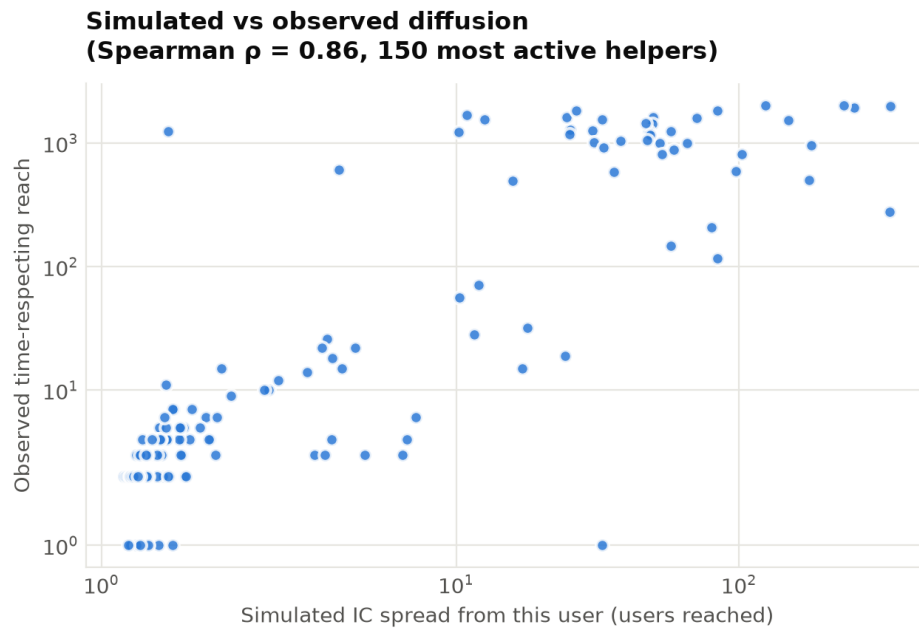


Figure 11. *Simulated vs observed reach (log scales).*

5.4 Answer prediction

Table 7 shows performance on the 792 newest questions, of which 59% were answered. All models clearly beat the baseline. The best, Random Forest, reaches ROC-AUC 0.675. This is in line with published results for answer prediction, which is hard because whether a question is answered also depends on who happens to be online. Cross-validation scores on the training period are lower (about 0.60), suggesting the newer period, with fewer and more predictable users, is somewhat easier. An ablation that added 16 principal components of the sentence embeddings raised ROC-AUC by only 0.007, so the simpler model was kept.

Table 7. *Answer prediction on future questions (temporal split)*

Model	ROC-AUC	PR-AUC	Accuracy	Bal. acc.	F1	CV ROC-AUC
Baseline (majority class)	0.500	0.591	0.591	0.500	0.743	0.500
Logistic Regression	0.673	0.725	0.624	0.626	0.657	0.593
Random Forest	0.675	0.725	0.610	0.629	0.613	0.602
XGBoost	0.656	0.715	0.639	0.600	0.727	0.599

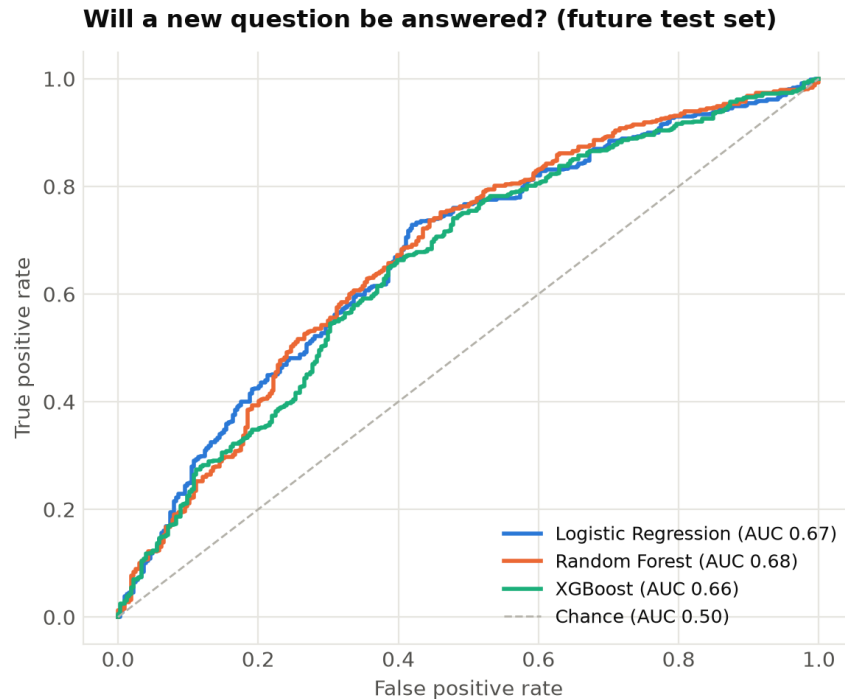


Figure 12. ROC curves on the future test set.

The most important features (Figure 13) are the topic's earlier answered rate, the number of tags, the asker's earlier answered rate, and whether the question contains code. The practical advice for learners follows directly: tag questions fully and include the relevant code.

What predicts an answer (Random Forest, permutation importance)

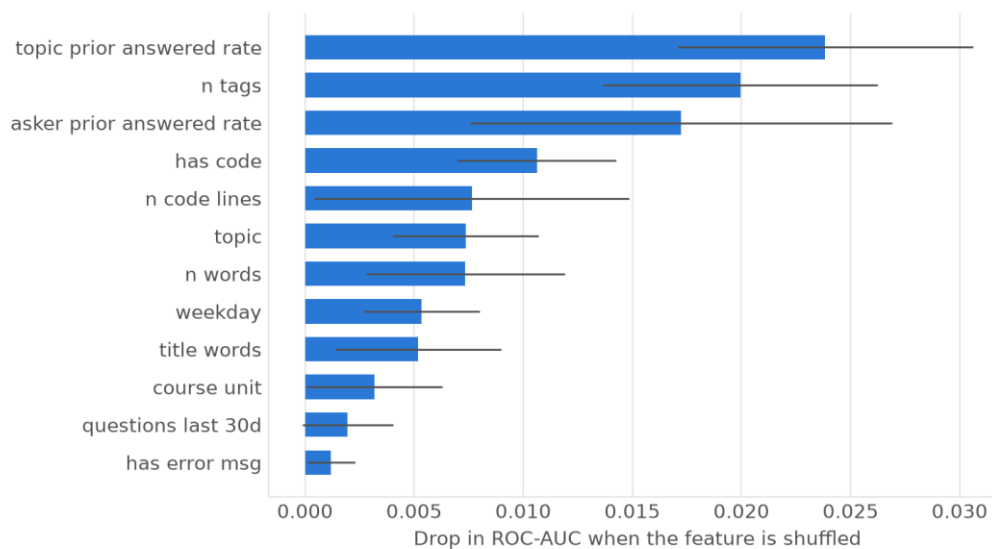


Figure 13. Permutation feature importance of the best model.

5.5 Expert recommendation

The hybrid expert finder with sentence embeddings is the best method: a real answerer appears in its top 10 for 81.1% of future questions and in its top 5 for 72.5% (MRR 0.637). The popularity baseline is strong (70.2% Hit@10) because one expert answers a very large share of questions, but it cannot go

beyond the few most active people. Adding recency and authority to content similarity gives the largest gain at $k = 10$.

Table 8. Expert finder evaluation on future questions

Method	Hit@1	Hit@5	Hit@10	MRR
Content (tfidf)	52.4%	68.9%	71.0%	0.596
Hybrid (tfidf)	52.8%	71.7%	79.8%	0.628
Popularity (most answers)	47.2%	70.2%	70.2%	0.582
Content (embedding)	53.0%	70.8%	72.7%	0.611
Hybrid (embedding)	53.6%	72.5%	81.1%	0.637

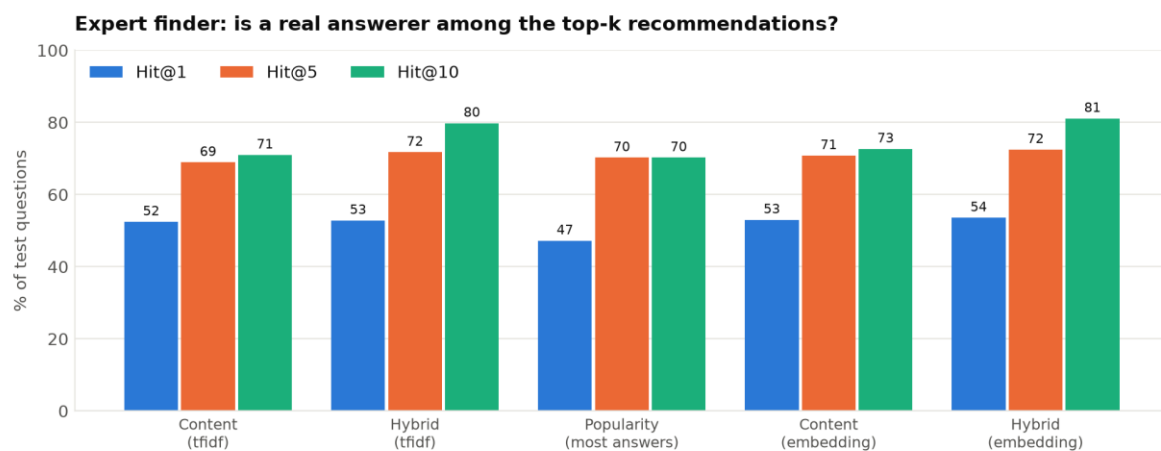


Figure 14. Share of future questions whose real answerer is in the top-k recommendations.

5.6 Interactive dashboard

All results are available in a seven-page Streamlit dashboard:

- **Overview:** key numbers and findings
- **Topics:** interactive topic map, difficulty and trends
- **Knowledge network:** a draggable network graph, top experts, roles and validation
- **Knowledge diffusion:** exposure test, adoption curves and influence maximisation
- **Answer prediction:** a “check your question” tool that estimates the chance of an answer and gives tips
- **Expert finder:** type a problem and get recommended experts and similar past questions
- **Database:** runs the SQL analysis queries against PostgreSQL

6. Discussion

6.1 What the results mean

For instructors. The difficulty index points to where support is most needed: Tideman and the web-development problem sets combine high unanswered rates with long waits. Targeted material, such as walkthroughs, office hours and FAQ posts, for these units would reach the learners who are struggling most.

For platforms. The community depends on very few people. If the top handful of experts left, most questions would go unanswered, and the decline after 2020 shows how quickly activity can shrink. Recognising and supporting core experts, and routing questions to less-loaded helpers through the expert finder, would reduce this risk.

On knowledge diffusion. Peer contact matters, but mainly for advanced topics; beginners mostly arrive at topics independently, driven by the course schedule. This suggests that social features such as study groups and mentoring pay off most in the later part of a course.

6.2 Limitations

- The sample covers the 4,000 newest questions; very old activity and users' history outside the sample are missing, which weakens comparisons with site-wide reputation.
- Adoption is inferred from posting, not from reading, so learners who read without posting are invisible.
- The shuffle test controls for timing, not for all forms of homophily; the exposure effect is evidence consistent with influence, not proof of causation.
- IC/LT parameters ($\beta = 0.1$, weighted thresholds) are standard choices, not fitted values; the strong correlation with observed reach supports them but does not calibrate them.
- The answer predictor assumes information that is available at posting time; its accuracy is limited by factors the data cannot see, such as who is online.

6.3 Lessons learned

Validation against independent evidence was essential. Three bugs in the first diffusion analysis produced impressive but false results: an exposure share above 100%, a strong learner-to-helper effect, and near-deterministic simulation. They were found only because every result was checked for plausibility and against a second source. Leakage-free features and temporal splits likewise give lower but honest scores.

7. Conclusion and Future Work

This project built a complete learning-community analytics system, from API data collection and a PostgreSQL database to NLP, network analysis, diffusion modeling, two predictive applications and an interactive dashboard, and applied it to 4,000 CS50 questions. BERTopic recovered the course syllabus from learners' own words and identified the hardest units. Network analysis showed that a tiny core of experts sustains the community. Diffusion analysis found a modest, statistically significant social-exposure effect concentrated in advanced topics, and a simulation that agrees with observed spread ($p = 0.86$). The answer predictor (ROC-AUC 0.675) and expert finder (Hit@10 81%) turn these insights into tools.

Future work

- Extend to several communities (for example Data Science and Statistics Stack Exchange) to compare course-based and open communities.
- Use an LLM to name topics and summarise what learners find confusing in each one.
- Fit diffusion parameters from observed cascades instead of fixing them.
- Deploy the expert finder as a live service that suggests helpers when a question is posted, and measure its effect on answer rates.
- Add dropout prediction: use network position and topic difficulty to flag learners at risk of leaving.

References

- [1] D. M. Blei, A. Y. Ng and M. I. Jordan, “Latent Dirichlet Allocation,” *Journal of Machine Learning Research*, vol. 3, pp. 993–1022, 2003.
- [2] M. Grootendorst, “BERTopic: Neural topic modeling with a class-based TF-IDF procedure,” *arXiv:2203.05794*, 2022.
- [3] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in *Proc. EMNLP-IJCNLP*, 2019.
- [4] L. McInnes, J. Healy and J. Melville, “UMAP: Uniform Manifold Approximation and Projection for dimension reduction,” *arXiv:1802.03426*, 2018.
- [5] R. J. G. B. Campello, D. Moulavi and J. Sander, “Density-based clustering based on hierarchical density estimates,” in *Proc. PAKDD*, 2013.
- [6] G. Bouma, “Normalized (pointwise) mutual information in collocation extraction,” in *Proc. GSCL*, 2009; M. Röder, A. Both and A. Hinneburg, “Exploring the space of topic coherence measures,” in *Proc. WSDM*, 2015.
- [7] A. B. Dieng, F. J. R. Ruiz and D. M. Blei, “Topic modeling in embedding spaces,” *Transactions of the ACL*, vol. 8, pp. 439–453, 2020.
- [8] J. Zhang, M. S. Ackerman and L. Adamic, “Expertise networks in online communities: Structure and algorithms,” in *Proc. WWW*, 2007.
- [9] V. D. Blondel, J.-L. Guillaume, R. Lambiotte and E. Lefebvre, “Fast unfolding of communities in large networks,” *Journal of Statistical Mechanics*, P10008, 2008.
- [10] A. Anderson, D. Huttenlocher, J. Kleinberg and J. Leskovec, “Discovering value from community activity on focused question answering sites: A case study of Stack Overflow,” in *Proc. KDD*, 2012.
- [11] D. Kempe, J. Kleinberg and É. Tardos, “Maximizing the spread of influence through a social network,” in *Proc. KDD*, 2003.
- [12] J. Leskovec, A. Krause, C. Guestrin, C. Faloutsos, J. VanBriesen and N. Glance, “Cost-effective outbreak detection in networks,” in *Proc. KDD*, 2007.
- [13] A. Anagnostopoulos, R. Kumar and M. Mahdian, “Influence and correlation in social networks,” in *Proc. KDD*, 2008.
- [14] M. Asaduzzaman, A. S. Mashiyat, C. K. Roy and K. A. Schneider, “Answering questions about unanswered questions of Stack Overflow,” in *Proc. MSR*, 2013.
- [15] X. Liu, W. B. Croft and M. Koll, “Finding experts in community-based question-answering services,” in *Proc. CIKM*, 2005.

Appendix A. Running the System

The source code, data and results are in the project's GitHub repository. On a new Windows laptop:

1. Install Python 3.12, PostgreSQL (with pgAdmin) and Git.
2. In pgAdmin, create a database named learning_community.
3. git clone the repository, then run: `pip install -r requirements.txt`
4. Copy `.env.example` to `.env` and enter the PostgreSQL password.
5. `python src/load_to_postgres.py` (creates 16 tables with the data and results).
6. `streamlit run app/streamlit_app.py` (opens the dashboard at `http://localhost:8501`).

Optional: `python src/run_pipeline.py` re-runs the whole analysis (about 10 minutes); `python src/collect_data.py` downloads fresh data from the Stack Exchange API.

Tools and libraries

Python 3, pandas, scikit-learn, sentence-transformers, BERTopic, UMAP, HDBSCAN, NLTK, NetworkX, XGBoost, SciPy, Matplotlib, Plotly, PyVis, Streamlit, SQLAlchemy, psycopg2 and PostgreSQL.